

Project spec — Does perp flow and positioning improve short-horizon spot prediction?

1. Overview and research question

Rubric: problématique and contextualisation (Démarche scientifique); sets the thread every later section must serve.

Research question: Does perpetual futures flow and positioning improve short-horizon BTC spot price prediction over a spot-only baseline?

Scope: BTC is the headline asset. ETH, SOL, BNB, ZEC and DOGE extend the test across liquidity levels, which answers a sub-question: *does perp information matter more for less liquid coins?* All six trade as spot and USD-M perpetual pairs (USDT-quoted) on Binance; verify this in the notebook (Section 3).

Motivation: in crypto, a large share of price discovery is thought to happen on perpetual futures rather than spot. Perps also carry information spot cannot: leveraged positioning (open interest), the cost of holding positions (funding), and the perp-spot premium (basis). If that information reaches spot with a delay, it should improve forecasts.

Hypotheses:

1. H1: adding perp features to a spot-only model improves out-of-sample forecasts of spot returns for BTC.
2. H2: the improvement shrinks as the horizon lengthens (1 min → 1 h).
3. H3: the improvement is larger for less liquid coins (ZEC, DOGE) than for BTC and ETH.
4. H4: the improvement is larger in high-volatility regimes than in calm ones.

What a good answer looks like: a horizon-by-coin profile of the forecast gain, with significance tests, not a single accuracy number. A null result at some horizons is a valid finding if the design is sound.

Open question: if the cross-coin results become central, consider rewording the title to "BTC and major altcoins" so the question matches the scope.

2. Grading, deliverables and rubric cross-reference

Two graded deliverables, per the instructor's email: the **repo + PDF report**, due

Nov 24, 2026 at 23:59 Paris time, worth roughly 2/3 to 3/4 of the grade; and the **defense** on Dec 1, 2026, worth the rest (exact split not yet fixed).

Deliverable	What is required	Spec sections
Code repository (any Git platform, public)	All code needed; runs as-is when cloned by the grader (Fernanda); downloading reasonably sized data from inside the code is fine; heavy loads discouraged	3, 4, 11
PDF report in the repo	Goal and scope, methods, results	1, 5–10
Submission	Link to the public repo sent by email	13
Defense	At most 10 min of slides, uninterrupted; then free Q&A on methods, results and extensions, including tests of our understanding of our own code	10, 13

The [course evaluation page](#) categories remain our quality checklist for what a good repo and report contain:

Course category	Spec sections that earn it
Données (collecte et nettoyage)	3 Data sources, 4 Cleaning and storage, 6 Features
Analyse descriptive	7 Descriptive analysis, 10 Visualisation
Analyse et modelling	8 Models, 9 Training and testing
Démarche scientifique et reproductibilité	1 Question, 5 Horizons, 9 Testing, 11 Reproducibility, 12 Limitations
Format du code	11 Repo structure and Git workflow
Soutenance	10 Visualisation (slides), 13 Timeline

Hard requirements:

- ☐ Public repo with frequent commits and pull requests, not a single upload.
- ☐ A fresh clone runs end to end with no manual steps beyond installing dependencies; data downloads happen inside the code.
- ☐ Download size kept reasonable (Section 3); no data files committed to Git.
- ☐ `report.pdf` committed in the repo: goal and scope, methods, results.
- ☐ Dependencies installable from `pyproject.toml` or `requirements.txt`.
- ☐ Clean code: functions in `.py` modules, no copy-pasted cells.

- ☐ Every table and figure in the report is commented and tied back to the question.
- ☐ Every team member can explain any part of the code at the defense. AI use is fine; AI use without understanding is penalised.

Staying on topic: each section below opens with a **Rubric** line. If a planned analysis cannot name both a rubric category and a hypothesis (H1-H4) it serves, cut it.

3. Data sources and collection

Rubric: Données (collecte) — automated, multi-dataset collection from two access routes (static archive + REST API).

All data comes from Binance public endpoints, downloaded by the code itself, for six spot pairs and six USD-M perpetual pairs: BTCUSDT, ETHUSDT, SOLUSDT, BNBUSDT, ZECUSDT, DOGEUSDT. No API key is needed.

Design choice: 1-minute klines instead of tick data. Binance klines already contain the taker-buy volume per minute, so signed flow and VWAP can be computed without downloading every trade. This keeps a fresh clone fast and light, as the grader requires.

$$\text{signed volume} = 2 \times \text{taker buy volume} - \text{volume}$$

$$\text{VWAP} = \frac{\text{quote volume}}{\text{volume}}$$

Sample window (proposal): 2026-01-01 to 2026-06-30, about 6 months. Confirm every coin has both series over the full window before fixing it; hardcode the dates in `config.yaml` so results never shift between runs.

Dataset	Market	Route	Resolution	Used for
klines 1m	Spot	Archive	1 min	Spot VWAP returns (target), spot signed flow, volume, trade count
klines 1m	USD-M perp	Archive	1 min	Perp VWAP, perp signed flow
premiumIndexKlines 1m	USD-M perp	Archive	1 min	Basis / premium
metrics	USD-M perp	Archive	≈5 min (verify)	Open interest, long/short and taker ratios
liquidationSnapshot	USD-M perp	Archive	Event (verify completeness)	Liquidation flow

Dataset	Market	Route	Resolution	Used for
<code>fundingRate</code>	USD-M perp	REST API <code>/fapi/v1/fundingRate</code>	8 h (verify per symbol)	Funding level and changes
First candle / exchange info	Spot + perp	REST API	—	Availability check per coin
<code>aggTrades</code> , a few days of BTC only	Spot + perp	Archive	Tick	Optional second- level lead-lag figure (Section 7)

Download size: klines are one small monthly file per symbol and market, so the full set should stay modest. Measure the actual total after the first download and record it in the README. If it is heavier than expected, shorten the window or add a `--quick` mode (BTC only, one month) for a fast check.

Routes:

- **Archive** (`data.binance.vision`): static zipped CSVs with a `.CHECKSUM` per file. Bulk history, no rate limits, identical files on every download.
- **REST API** (`api.binance.com`, `fapi.binance.com`): funding history and availability checks. Responses are cached locally. Open-interest and ratio statistics via REST only cover the latest 30 days, so historical positioning comes from the archive `metrics` files.

Collection steps (all run by `python -m src.pipeline`):

1. Availability check: earliest spot and perp candle per coin (REST); output a table for the report.
2. Build archive URLs from the config (market, dataset, symbol, month); download, verify checksums, unzip into the gitignored `data/` folder; skip files already present.
3. Fetch funding history per symbol with pagination (REST); cache raw JSON.
4. Log every download (file, size, checksum status) so provenance is documented.

4. Data cleaning, structuring and storage

Rubric: Données (nettoyage) and reproductibilité — documented cleaning choices, raw → clean pipeline, data stored outside Git.

The pipeline has three layers, each saved as parquet in the gitignored `data/` folder so later steps never re-download:

1. **Raw:** files exactly as downloaded (zipped CSV, raw JSON from the API).
2. **Clean:** typed, deduplicated, one schema per dataset, timestamps in UTC milliseconds.
3. **Panel:** one 1-minute table per coin, spot and perp aligned on the same timestamp index.

Cleaning tasks (document each choice in the report):

- **Headers and units:** archive files have inconsistent headers across markets and periods; spot timestamps switch from milliseconds to microseconds on 2025-01-01. Normalise to one unit.
- **Gaps:** flag missing minutes (exchange maintenance, quiet minutes on thinner coins like ZEC); keep a gap mask rather than silently forward-filling; drop windows where spot or perp is missing.
- **Derived columns:** per minute and market — VWAP (quote volume / volume), signed volume ($2 \times \text{taker buy volume} - \text{volume}$), trade count.
- **Lower-frequency joins:** `metrics` (≈ 5 min) and funding (8 h) are joined to the 1-min grid **as of the last value known before each minute** (backward as-of join), never forward. This is the main leakage guard at the data stage.
- **Outliers:** inspect extreme returns (fat-finger prints, listing effects); winsorise features if needed, never the target.
- **Validation:** spot and perp close prices should track each other closely; large persistent gaps flag a join or unit bug.

Storage:

- No data in Git: `data/` is in `.gitignore`; the code regenerates everything from public Binance URLs.
- Local parquet cache, so re-running the notebook after the first download takes seconds.
- Tools: pandas throughout (1-min data for 6 coins is small); DuckDB or polars only if a step turns out slow.

5. Time horizons and prediction targets

Rubric: Démarche scientifique — justified horizon choice, target defined to avoid microstructure artefacts; tests H2.

All work runs on a **1-minute base grid**; we forecast at four horizons: **1 min, 5 min, 15 min and 1 h**.

Horizon	Observations per coin (6 months)	Perp features usable	Note
1 min	$\approx 260,000$	Signed flow, basis	Highest power, most microstructure noise

Horizon	Observations per coin (6 months)	Perp features usable	Note
5 min	≈ 52,000	+ open interest, ratios (metrics)	Check metrics resolution
15 min	≈ 17,000	Same	Balance of signal and sample size
1 h	≈ 4,300	Same + funding	Lowest power

Observation counts are for non-overlapping windows; forecasts are issued every minute, so labels overlap (handled in Section 9).

Why not seconds: the spot archive has no historical best bid/ask, so the target is trade-based. At second scale, trades bounce between bid and ask, creating spurious negative autocorrelation. Spot-perp arbitrage also closes gaps within milliseconds. An optional BTC-only second-level cross-correlation (Section 7) shows this visually.

Why not days: too few observations to detect small effects, and the derivatives-lead-spot mechanism is short-horizon.

Targets: the forward spot log return over horizon h , computed from 1-min VWAPs.

$$y_{\{t,h\}} = \log \mathrm{VWAP}^{\mathrm{spot}}_{\{t+h\}} - \log \mathrm{VWAP}^{\mathrm{spot}}_{\{t\}}$$

- **Regression target:** the return itself.
- **Classification target:** three classes — up / flat / down — with a threshold per coin and horizon (e.g. a fixed fraction of the rolling volatility), so "flat" absorbs near-zero noise. Fix the threshold on training data only.

6. Features

Rubric: Données (variables créées) and Démarche scientifique — the two feature blocks are the experiment for H1.

Features are split into two blocks. The baseline model uses block A only; the full model uses A + B. Every feature at time t uses data strictly before t .

Block	Feature	Definition (computed over lags of 1, 5, 15, 60 min)	Source
A — spot only	Lagged returns	Past spot VWAP log returns	Spot klines

Block	Feature	Definition (computed over lags of 1, 5, 15, 60 min)	Source
A — spot only	Spot signed flow	Sum of signed volume / total volume	Spot klines
A — spot only	Realised volatility	Rolling std of 1-min returns	Spot klines
A — spot only	Activity	Trade count and volume vs. rolling average	Spot klines
A — spot only	Time of day	Hour-of-day encoding (crypto trades 24/7 but activity is cyclical)	Timestamp
B — perp	Perp signed flow	Signed volume / total volume on the perp	Perp klines
B — perp	Flow divergence	Perp signed flow minus spot signed flow	Both
B — perp	Basis	(Perp VWAP – spot VWAP) / spot VWAP, in bps; and its change	Both / premium index
B — perp	Perp lead return	Past perp return minus past spot return	Both
B — perp	Open interest change	% change in open interest since last reading	metrics
B — perp	Taker / long-short ratios	Latest available ratios	metrics
B — perp	Liquidations	Liquidated notional by side	liquidationSnapshot
B — perp	Funding	Last settled funding rate	Funding history

Rules:

- **Normalise across coins:** basis in bps, flows as fractions of volume, OI as % changes, so pooled models compare like with like.
- **Respect resolution:** positioning features (≈ 5 min) and funding (8 h) are only claimed to help at horizons $\geq$ their own resolution.
- **Altcoin control:** for ETH, SOL, BNB, ZEC and DOGE, add lagged BTC spot returns to block A, so perp effects aren't just BTC leading the market.

- **Feature count discipline:** start with a small set (about 10–15 per block), justify each, add more only if a hypothesis needs it.

7. Descriptive analysis

Rubric: Analyse descriptive (4 pts) — every statistic is interpreted and linked to H1–H4; no uncommented `describe()`.

The descriptive part must give a first answer before any model: *is there visible lead-lag from perp to spot, and where?*

Analysis	What it shows	Hypothesis
Data summary per coin: trades/day, volume, share of perp vs spot volume, missing minutes	Liquidity differences across the six coins; where perp dominates	H3, data quality
Return distributions per horizon (moments, tails)	Fat tails, why plain accuracy is misleading, why we need a flat class	Target design
Basis over time and its distribution	How far and how long perp deviates from spot; spikes during stress	H1, H4
Funding and open interest over time	Leverage build-up periods, link to later price moves	H1, H4
Cross-correlation of perp flow vs. future spot returns, lags –10 to +10 min	Direction and length of the lead-lag; does it vanish beyond a few minutes?	H1, H2
Same cross-correlation, per coin	Is lead-lag longer for less liquid coins?	H3
Granger causality tests, both directions	Does perp predict spot, spot predict perp, or both?	H1
Volatility regimes (rolling vol terciles) and liquidation clusters	Defines calm vs stress periods used later	H4
Optional: BTC second-level cross-correlation (a few days)	Shows lead-lag at the sub-second-to-second scale and justifies minute horizons	H2

Interpretation standard: each item ends with one or two sentences: what we see, what it suggests for the hypotheses, and any caveat (e.g. Granger causality is predictive, not causal).

8. Models

Rubric: Analyse et modelling — models chosen to answer the question, justified, interpreted; not a leaderboard.

The core design is an **ablation**: each model is fitted twice, on block A (spot-only baseline) and on blocks A + B (with perp features). The gain from A to A + B is the answer to H1.

Level	Model	Library	Why it's here
0	Zero-return forecast (random walk); majority class	numpy	The bar to beat; short-horizon returns are close to unpredictable
1 (core)	Ridge / lasso regression on returns; multinomial logistic regression on up/flat/down	scikit-learn, statsmodels	Interpretable coefficients and HAC inference: says <i>which</i> perp variables matter
2	Gradient boosting (<code>HistGradientBoostingRegressor</code> / <code>Classifier</code> , or LightGBM)	scikit-learn / lightgbm	Checks for non-linear effects and interactions (e.g. basis only matters when funding is extreme)
3 (optional)	HAR-RV on realised volatility, with and without perp features	statsmodels	Secondary question: do OI and funding help forecast volatility? Reuses the GARCH/ARCH idea where it fits

Variants:

- **Per-coin models** for each of the six coins.
- **Pooled model** across coins with coin fixed effects and interaction terms (perp features × liquidity) to test H3 directly.

Deliberately excluded (state this in the report): polynomial regression (overfits non-stationary prices, no motivation here) and deep learning (LSTM, TLOB-style transformers): hard to justify at this data scale and would turn the project into a benchmark race.

Interpretation outputs: coefficient tables with HAC standard errors (level 1); permutation importance of block-B features (level 2). Treat importances as descriptive, not causal.

9. Training, validation and statistical testing

Rubric: Démarche scientifique — leakage-free validation and significance tests turn "the score went up" into a defensible claim.

Walk-forward validation (no random splits):

1. Split the 6-month window into monthly blocks.
2. Fold k : train on all months before month k , test on month k (expanding window). First test month = month 3, giving 4 test folds.
3. Tune hyperparameters (ridge/lasso penalty, boosting depth and learning rate) on the last month of each training window only.
4. Leave an **embargo** gap of at least h between train and test so overlapping labels can't leak.
5. Fit scalers, class thresholds and feature selection on training data only.

Metrics (computed on test folds, per coin $\times$ horizon):

- Regression: out-of-sample R^2 relative to the zero-return forecast (negative = worse than doing nothing).
- Classification: balanced accuracy, macro F1 and log loss (accuracy alone is misleading when "flat" dominates).

Significance of the perp gain:

- **Clark-West test:** the standard test for nested models (A is nested in $A + B$). This is the main test for $H1$.
- **Diebold-Mariano test:** for non-nested comparisons (e.g. linear vs boosting).
- **HAC (Newey-West) standard errors** for coefficients, since returns and overlapping labels are autocorrelated.
- **Multiple testing:** 6 coins $\times$ 4 horizons $\times$ several models = many tests. Report a pooled test as the headline and apply a Holm correction to per-cell results, or label them exploratory.

Stress and heterogeneity checks:

- Re-compute the gain separately in calm vs high-volatility terciles ($H4$).
- Compare gains across coins ranked by liquidity ($H3$).

Optional economic check: a simple long/short rule on the forecasts, net of trading fees and spread (funding is negligible at these horizons unless a position is held through a settlement). Only include it if time allows and costs are modelled honestly.

10. Visualisation

Rubric: Analyse descriptive (graphical part) and Soutenance — the right chart type for each message, titled axes and legends, every figure commented.

The report keeps about 8–10 figures; extras go to an appendix. The four marked ★ carry the oral presentation.

#	Figure	Chart type	Message
1	Data coverage per coin	Heatmap (coin × day, missing minutes)	Data quality and gaps
2	Perp vs spot volume share per coin	Bar chart	Where perp dominates price discovery
3	Basis over time, with liquidation events marked	Line chart, one panel per coin	Size and timing of perp-spot deviations
4 ★	Cross-correlation of perp flow vs future spot returns by lag	Line chart with confidence band, one line per coin	Lead-lag exists and fades within minutes
5	Return distribution vs normal	Histogram / QQ plot	Fat tails justify the three-class target
6 ★	Forecast gain (A + B vs A) by coin × horizon	Heatmap with significance markers	Main result: where perp features help
7 ★	Gain vs horizon	Line chart, one line per coin, error bars across folds	H2: the gain shrinks with horizon
8	Gain in calm vs volatile regimes	Grouped bar chart	H4
9	Coefficients / permutation importance of perp features (BTC)	Horizontal bar chart with HAC intervals	Which perp signals matter
10 ★	Gain vs coin liquidity	Scatter plot	H3

Conventions:

- One colour per coin, fixed across every figure; colour is never the only carrier of meaning.
- Units on every axis (bps, %, minutes); dates in UTC.
- Libraries: matplotlib/seaborn for static figures; plotly for interactive ones in the notebook.
- Optional: a small Streamlit app to browse coin × horizon results; only if the core notebook is finished (the course treats it as a bonus, not a requirement).

Defense slides (10 minutes, no interruptions): about 8 slides, roughly one minute each.

1. The question and why perps might lead spot.
2. Data: sources, six coins, window, what was built from klines.
3. Descriptive evidence: figure 4 (lead-lag cross-correlation).
4. Method: the ablation, walk-forward validation, Clark-West test.
5. Main result: figure 6 (gain by coin × horizon).
6. Horizon and liquidity profile: figures 7 and 10.
7. Limitations and what we would do next.
8. Backup slides (not presented): feature definitions, leakage checks, per-coin tables, to answer Q&A quickly.

Rehearse to 9 minutes; the Q&A will probe methods, extensions and our own code.

11. Reproducibility, repo structure and Git workflow

Rubric: the repo + report deliverable (about 2/3 to 3/4 of the grade) — Démarche scientifique et reproductibilité and Format du code. The grader clones the repo and runs it; anything that breaks there costs the most.

Repo layout:

```
project/
├─ README.md           # question, one-command run, expected download size
and runtime
├─ report.pdf          # the graded report: goal and scope, methods,
results
├─ pyproject.toml      # pinned dependencies (uv or pip)
├─ config.yaml         # coins, dates, horizons, paths
├─ .gitignore          # data/, caches, .env
├─ src/
│   └─ pipeline.py     # entry point: fetch → clean → features → models →
figures
```

```

|   ├── fetch.py           # archive download, checksums, REST calls + cache
|   ├── clean.py          # raw → clean → 1-min panel
|   ├── features.py       # blocks A and B, as-of joins
|   ├── models.py         # ablation fits per coin / pooled
|   ├── evaluate.py       # walk-forward, metrics, Clark-West, DM
|   └── plots.py          # one function per figure
└── notebooks/
    └── analysis.ipynb     # runs top to bottom, imports src/
└── report/
    └── report.qmd         # Quarto source rendered to report.pdf
└── slides/               # defense deck
└── tests/                # small checks: no-lookahead, timestamp units

```

Reproducibility rules:

- One command after installing dependencies (`python -m src.pipeline`) downloads the data from public Binance URLs, builds every table and figure, and saves them for the report.
- Fixed sample window and random seeds in `config.yaml` ; a `--quick` flag (BTC, one month) for a fast check.
- README states the expected download size and runtime, measured on a fresh clone.
- No secrets needed (public endpoints). If a key is ever added, it goes in `.env` , never in code.
- A no-lookahead unit test: changing data after t must never change features at t .
- The report is written in Quarto from the same saved figures, then rendered to `report.pdf` and committed, so text and code agree.
- Before Nov 24: fresh clone on a different machine, fresh environment, run everything.

Git workflow:

- One branch per task, merged through pull requests reviewed by a teammate.
- Small, frequent commits with clear messages.
- Only one person edits the notebook at a time; strip notebook outputs before committing (e.g. `nb-clean`) to avoid merge conflicts.
- Reviews double as defense prep: each reviewer must be able to explain the code they approve.

12. Risks, limitations and fallbacks

Rubric: Démarche scientifique — naming limits honestly is part of a scientific approach; fallbacks protect the reproducibility grade.

Risk	Impact	Mitigation / fallback
Repo fails when the grader clones and runs it	Most of the grade lost	One-command pipeline, README with exact steps, fresh-clone test on another machine before Nov 24
Download too heavy for the grader	Run abandoned or penalised	1-min klines instead of tick data; measure size early; <code>--quick</code> mode; shorten window if needed
A team member can't explain part of the code at the defense	Flagged as vibe-coded	Every PR reviewed and explained by a teammate; walk-through session of all modules before Dec 1
Archive dataset missing or incomplete for some coin/period (<code>metrics</code> , <code>liquidationSnapshot</code>)	Block B partly empty	Check coverage in week 1; drop the feature for that coin and say so
Leakage (same-time perp and spot prices)	Fake, inflated results	As-of joins, embargo, no-lookahead test; results that look too good = bug
Null result	Weak-looking conclusion	Hypotheses fixed in advance and a horizon profile make a null result informative; report effect sizes with intervals
Binance changes archive format or endpoints	Download step breaks	Pin URLs and schemas in config; checksum checks fail loudly
Scope creep (TLOB, options, Streamlit)	Core unfinished by Nov 24	Extensions only if the core runs end to end before the Nov 16 freeze

Limitations to state in the report:

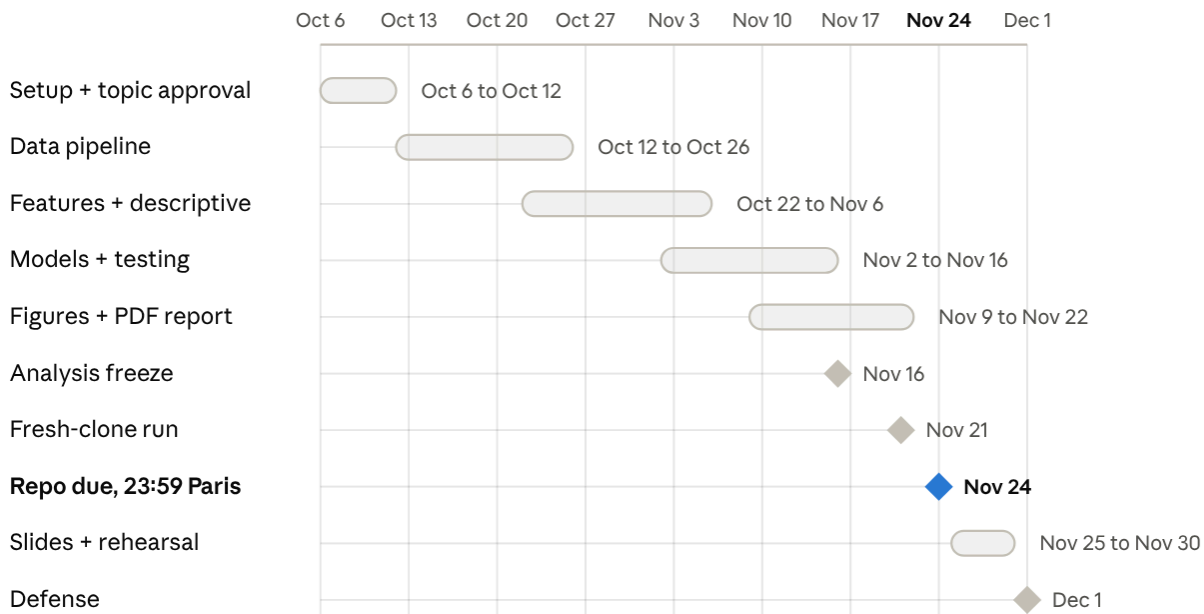
- Single venue: Binance flow may not represent the whole market.
- Trade-based target, no historical spot bid/ask, so some microstructure noise remains at 1 min.
- Predictive results, not causal ones (including Granger tests).
- Forecast gains need not be tradable after costs.
- The sample period's regime (trend, volatility) may not generalise.

13. Timeline and team split

Rubric: both deliverables — the repo + report (fresh-clone run, collaborative Git history) and the defense (time to build slides and rehearse the Q&A).

Repo due Nov 24, then a week to prepare the Dec 1 defense

Proposed plan by week, 2026; analysis freezes Nov 16 to leave time for checks



project timeline · 6 phases, 4 milestones

The schedule is tight: seven weeks from today to the repo deadline. Phases overlap so nobody waits — features start on the first cleaned month while the pipeline finishes the rest. The analysis freezes on Nov 16; the last week before submission is for the PDF report, the fresh-clone test and fixes. Send the public repo link by email before Nov 24, 23:59 Paris time.

First actions (this week):

- ☐ Get the topic validated by the chargé de TD.
- ☐ Create the GitHub repo, `pyproject.toml` and `config.yaml`.
- ☐ Run the availability check for the six coins and fix the sample window.
- ☐ Download one month of BTC data end to end to size the pipeline.

Team split (by pipeline stage, not by coin):

Team Member	Role	Owns	Spec sections
Antoine	Data & Project Lead	<code>fetch.py</code> , <code>clean.py</code> , storage, availability check; project admin: repo setup, README, topic approval, deadlines, sending the repo link	1, 2, 3, 4, 11, 13
Tina	Analysis Lead	<code>features.py</code> , descriptive analysis, figures, risks and limitations	6, 7, 10, 12
Arya	Modelling Lead	<code>models.py</code> , <code>evaluate.py</code> , statistical tests	5, 8, 9

Everyone reviews another member's pull requests, and all three co-write the report. Because the Q&A can test any of us on any part of the code, hold a walk-through of every module in the week before the defense, where each person explains a module they did not write.